J. Software Version Control with Git

Appendix J of the EFC Gases & Advanced Materials Controls Standard, Revision H, October 2, 2026. Standalone copy; see the full document for the other chapters and appendices.

[Future: this appendix describes the planned Git integration from Section 7.2 of the Controls Standard. It is not current practice. Until it is adopted, the manual file-naming rules in Section 7.2 of the Controls Standard apply.]

J.1 Git Basics for Controls Engineers

Git is a version control tool. It keeps the full history of a set of files in a *repository* ("repo"), so you can see what changed, when, by whom, and why, and go back to any earlier version.

- **Commit:** a saved snapshot of the files with a short message describing the change. Commit at meaningful points, not every save.
- **Tag:** a permanent label on a commit. Tags mark revisions: `rev-C` during development, `rev-0` at turnover, `rev-1` and later after turnover.
- **Branch:** a parallel line of work, used to try a change without disturbing the released version. Most projects need only a main branch.
- **Remote:** a shared copy of the repo on a company server (for example GitHub, GitLab, or Azure DevOps, as chosen with IT), so the history is not lost with one laptop.
- **.gitignore:** a list of files Git should not track (temporary, generated, or machine-specific files).

How the EFC rules map onto Git:

- One repository per project (for example `014_R23-Column`), holding the PLC and SCADA folders.
- Lettered development revisions and Rev 0 become tags; the date suffix in file names is no longer needed because Git records the date of every commit.
- A change made under MOC is committed with the MOC number in the message (for example `MOC-2027-014: raise PT-210 HH limit`) and tagged as a new revision.
- The companion revision log (Section 7.2 of the Controls Standard) can be generated from the tag and commit messages.

J.2 Control Expert Classic

The problem. The normal project file (`.STU`) is binary, so Git can store it but cannot show what changed inside it. Control Expert has no built-in Git support.

The approach. Commit the binary project together with a full text export of the same version:

- `.STU` — the working project file (or `.STA`, the archive, which can only be created after the project is built).
- `.XEF` — a full project export in XML. It can be made at any stage and imported to rebuild the project (then saved as `.STU`, rebuilt, and downloaded). Because it is text, Git can show changes to logic, variables, and configuration.
- `.ZEF` — the XEF export plus the DTM (device) configuration; use it when the project has DTMs, such as EtherNet/IP devices.

XML differences are readable but noisy. Schneider's Control Expert DIF utility compares two exports more clearly, and third-party tools (for example Copia) provide Git-based version control with graphical LD/ST/FBD differences for Control Expert.[1]

Routine:

1. Save the project and build it.
2. *File* > *Export Project* to `.XEF` (or `.ZEF`) in the repo folder, using the same name as the project.
3. Commit both files with a message describing the change; tag it if it is a revision.

Exclude: `.BAK` automatic backups and temporary files.

J.3 Productivity Suite

The problem. The project file (`.adpro`) is also binary, and Productivity Suite has no Git support.

The approach. Commit the `.adpro` file together with the tag database CSV export (*Tag Database* > *Export*, which produces `_basic.csv` and `_extended.csv` files) so tag changes are visible in Git. Logic changes are not visible as text; use Productivity Suite's built-in **Project Compare** (available since v2.0) to compare two committed versions, and describe the logic change in the commit message.

Routine: save the project, export the tag database to the repo folder, commit both, and tag revisions. Reusable User Defined Instructions (UDIs) exported to the UDI library can be kept in a shared library repository.

J.4 Ignition 8.1

Ignition projects are stored on the gateway as folders of files, which suits Git well.

- **Where:** `<Ignition install>/data/projects/<project name>`. Views, scripts, named queries, and other resources are stored as folders containing `resource.json` plus content files (for example `view.json` or `code.py`). Create the repository in the project folder (or in `data/projects` for several projects).
- **Ignore:** the `.resources` folder and other generated files.
- **Pulling changes onto a gateway:** after a `git pull`, tell the gateway to pick up the files with `system.project.requestScan()` (gateway scope), or a scan endpoint module, so the Designer and clients see the update.
- **Not in the project folder:** gateway settings such as tag providers and tags, device connections, database connections, users, and alarm pipelines are stored in the gateway's internal database. Back

[1] Schneider Electric, Control Expert help, *User Application and Project File Formats*; Copia Automation, *Schneider Control Expert* support documentation.

these up with a gateway backup (`.gwbk`), which is all-or-nothing, and export tags to JSON so tag changes can also be committed.

Inductive Automation recommends Git on the project folder to move work between development, test, and production gateways.[2]

J.5 Changes With Ignition 8.3

Ignition 8.3 (released September 16, 2025, a long-term support release) was designed with Git-based version control in mind:[3]

- **Whole-gateway configuration as files.** All gateway configuration moves from the internal database to files under `data/config` (for example tags under `data/config/resources/core/ignition/tag-definition`), so devices, tags, connections, and other settings can be committed along with projects.
- **Scan endpoints.** The REST API provides `/data/api/v1/scan/config` and `/data/api/v1/scan/projects`, and the gateway has a *Scan File System* button, to load pulled changes.
- **Deployment modes.** One configuration can hold settings for development, test, and production, selected on each gateway with the JVM argument `-Dignition.config.mode=<mode>`. Environment-specific values (addresses, credentials) no longer need separate copies of the project.
- **REST API** with a generated OpenAPI description at `/openapi`, and **secrets management** so passwords are not stored in committed files.
- **.gitignore template** provided by Inductive Automation; exclude `valueStore.idb` and some Vision binary resources.

What changes for us at the upgrade: the repository can cover the whole gateway, not just projects; gateway backups become a recovery tool rather than the only record of configuration; and moving a tested change from a development gateway to production becomes a Git pull plus a scan. Note that 8.3 no longer allows hot-swapping modules, so module installs still need a restart.

J.6 Checklists

Control Expert:

- ☐ Project saved and built; `.XEF`/`.ZEF` exported with the same name
- ☐ `.STU` (or `.STA`) and export committed together with a clear message
- ☐ Revision tagged; MOC number in the message where applicable

Productivity Suite:

- ☐ `.adpro` saved; tag database CSVs exported
- ☐ Logic change described in the commit message (checked with Project Compare)
- ☐ Revision tagged

Ignition:

[2] Inductive Automation, *Ignition 8 Deployment Best Practices* (8.1 documentation); `system.project.requestScan` (8.1 scripting reference).

[3] Inductive Automation, *Ignition 8.3 Version Control Guide* and *New in 8.3* documentation; *8.1 to 8.3 Upgrade Guide*.

- ☐ Project folder committed; `.gitignore` in place
- ☐ 8.1: tags exported to JSON and a gateway backup taken; 8.3: `data/config` committed
- ☐ After a pull, project (and 8.3 config) scan triggered and the change tested